

REPORT GENERATOR

AUTOMATED DOCUMENT CREATION WITH FLASK

ANTHONY SUNG

NATIONAL CHENGCHI UNIVERSITY

MAY 26, 2025

Table of Contents

- 1 Introduction
- 2 Project Overview
- 3 Business Value
- 4 Features
- 5 Implementation Details
- 6 Conclusion

INTRODUCTION

Challenges in Manual Report Generation

- Time-consuming, tedious, and repetitive
- Often leads to human-error
- Power BI, Tableau, or FineReport

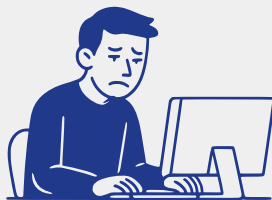


Figure 1: Tired Man

Why We Built This

- One tool for templates, data, and automation
- Less coding needed — just upload and go
- Built to work with **Google Drive** and charts

Smarter Reports, Less Effort



Figure 2: Easy Usage

- Upload templates and data
- Auto-fill variables and generate charts
- Preview and export .docx reports

How It Works: Tools & Technologies

- Backend: Python (Flask) + Pandas + DocxTpl
- Frontend: HTML + JavaScript + Google Picker API
- Visualization: Plotly + Kaleido
- Deployment: Render with OAuth and cloud integration

PROJECT OVERVIEW

Directory Structure

```
/.
├── Procfile .....Render deployment
├── README.md .....Project overview
├── app.py ..... Flask backend
├── credentials.json ..... OAuth token
├── fonts
│   └── cwTeXQYuan-Medium.ttf
├── requirements.txt ..... Python dependencies
├── static
│   ├── app-logo-no-text.svg
│   ├── autocomplete.min.js
│   ├── favicon.ico
│   ├── main.js ..... Frontend logic
│   ├── math.min.js
│   ├── picker.js ..... Google Drive Picker
│   └── style.css
└── templates
    ├── home.html
    ├── index.html ..... Upload page
    └── preview.html ..... Live preview
```

Step-by-Step Logic Flow

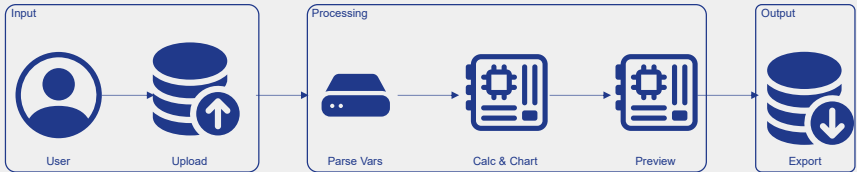


Figure 3: Report Generator Logic

Frontend: HTML + JS

- Upload templates and data files
- Display variable previews and chart selectors
- Integrate **Google Picker** for cloud storage access

Backend: Flask

- Parse placeholders in Word templates
- Process formulas using data from CSV/JSON
- Generate and insert charts into documents

Data Flow

1. User uploads '.docx' template and '.csv' data
2. Backend detects variables like `{{ revenue }}`
3. System evaluates formulas (e.g., `SUM(Revenue)`)
4. Chart is generated and saved as image
5. Final .docx report is rendered and ready for download

BUSINESS VALUE

reportGen Can Help!

- **Time & Cost Saving:** Reports that took an hour now finish in minutes saving hundreds of hours yearly across teams.
- **Reusable Templates:** Create once, reuse forever. Perfect for recurring reports with different data.
- **Fewer Errors:** Auto formulas and data validation reduce mistakes and improve consistency.
- **Cloud Ready:** Integrated with Google Drive and OAuth easy for remote and team-based work.

FEATURES

Word Templating with Placeholders

- ReportGen allows users to design Word templates using placeholders such as `{{ total_sales }}`
- These placeholders are detected and replaced with values dynamically derived from uploaded datasets
- Supports both scalar values and images (charts)

Data Upload and Variable Detection

- Supports CSV/Excel upload with drag-and-drop or Google Picker
- Automatically detects time columns and converts them to pandas datetime
- Variables in templates are cross-checked against uploaded data columns

Formula Engine

- Users can define formulas like `SUM(Sales)`, `MEAN(Revenue)`, or `PERCENT_CHANGE(Profit)`
- Formulas support chaining and nested calculations
- Dependencies between variables are resolved using topological sorting to ensure correct evaluation order

Dynamic Chart Generation

- Chart types supported: Line, Bar, Pie, and Histogram
- Charts are rendered using Plotly and exported as PNG via Kaleido
- Charts are then inserted into the Word document at predefined placeholders

Real-Time Document Preview

- Users can preview the rendered document before downloading it
- Variables with missing values are highlighted for user correction
- Interactive tooltips provide variable descriptions and chart information

Google Drive Integration

- Integrated with Google Picker API for file selection from Google Drive
- OAuth 2.0 authorization ensures secure access
- Files selected via Google Picker are loaded directly into the website

IMPLEMENTATION DETAILS

Flask: Lightweight, Powerful, Modular

- Routes handle file uploads, previews, and report generation
- Uses Jinja2 templates for dynamic HTML rendering
- Session-based design to manage files and variable states per user

Handling Requests the Flask Way

```
from flask import Flask, request, render_template, redirect, url_for
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return render_template('index.html')
```

```
@app.route('/submit', methods=['POST'])
def submit():
    data = request.form.get('data_field')
    return f"You submitted: {data}"
```

```
@app.route('/redirect-example')
def go_somewhere():
    return redirect(url_for('home'))
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Template Rendering: docxtp1

- Word templates use docxtp1 to render values into placeholders
- Scalar variables (e.g., total_sales) and charts (as images) are inserted dynamically
- Charts are saved as images and embedded using InlineImage

Handling Requests the Flask Way

```
from docxtpl import DocxTemplate, InlineImage
from docx.shared import Mm

# Load Word template
doc = DocxTemplate("template.docx")

# Prepare image and data
chart_img = InlineImage(doc, "chart.png", width=Mm(80))
context = {
    "title": "Monthly Sales Report",
    "chart": chart_img
}

# Render and export
doc.render(context)
doc.save("output.docx")
```

Chart Rendering: Plotly + Kaleido

- Charts are generated using Plotly based on user-selected configurations
- The Kaleido engine converts Plotly figures to static PNG files
- Chart files are inserted into Word at specified placeholders

Generating Line Chart with Plotly

```
import pandas as pd
import plotly.express as px

# Load data
df = pd.read_csv("sales_data.csv")

# Generate line chart
fig = px.line(
    df,
    x="date",
    y="sales",
    title="Monthly Sales Trend",
    labels={"date": "Date", "sales": "Sales Amount"}
)

# Export as PNG using Kaleido
fig.write_image("chart.png", engine="kaleido")
```

Formula Parsing and Evaluation

- Users can define formulas in settings or interactively.
- The parser supports nested formulas, dependencies, and chained evaluations.
- Topological sorting ensures correct evaluation order.

Formula Evaluation Example

```
import pandas as pd

# Sample dataset
df = pd.DataFrame({
    'sales': [100, 200, 300],
    'cost': [30, 80, 100],
})

# Evaluate 'profit' and 'profit_margin'
context = {}
context['profit'] = (df['sales'] - df['cost']).sum()
context['profit_margin'] = context['profit'] / df['sales'].sum()
```

Topological Sorting & Calculation

```
# Assuming topologically sorted order
ordered_vars = ['profit', 'profit_margin', 'summary']

# Final variable using previously computed results
context['summary'] = context['profit'] # Already summed above

print(context)
# {
#   'profit': 390,
#   'profit_margin': 0.65,
#   'summary': 390
# }
```


Google Picker API Integration

- Users can connect to their Google Drive to import 'docx', 'csv', and 'json' files
- Authentication is handled via OAuth 2.0 with secure token storage
- Picker interface allows real-time browsing of personal Drive folders

Google OAuth 2.0 Integration

```
from flask import Flask, redirect, session, request, url_for
from google_auth_oauthlib.flow import Flow

@app.route('/authorize')
def authorize():
    flow = Flow.from_client_secrets_file(
        'credentials.json',
        scopes=['https://www.googleapis.com/auth/drive.readonly'],
        redirect_uri=url_for('oauth2callback', _external=True)
    )
    auth_url, state = flow.authorization_url(prompt='consent')
    session['state'] = state
    return redirect(auth_url)
```

Handling Callback and Picker Token

```
@app.route('/oauth2callback')
def oauth2callback():
    flow = Flow.from_client_secrets_file(
        'credentials.json',
        scopes=['https://www.googleapis.com/auth/drive.readonly'],
        redirect_uri=url_for('oauth2callback', _external=True)
    )
    flow.fetch_token(authorization_response=request.url)

    credentials = flow.credentials
    session['access_token'] = credentials.token

    return render_template('picker.html', token=credentials.token)
```

Session Management and File Caching

- User uploads are stored temporarily in session-scoped folders
- Word and CSV files are cached for preview reuse
- On refresh, the system reuses cached content to avoid recomputation

CONCLUSION

Project Achievements

- Word template rendering with dynamic placeholders
- CSV data analysis and formula engine
- Chart generation and image embedding
- Google Drive file selection via Picker API

Challenges and Limitations

- Handling `eval()` securely in a sandboxed way
- Limited user control over advanced chart styling
- Current layout is not fully responsive on mobile devices
- Sessions are temporary; no persistence across visits

Future Development Directions

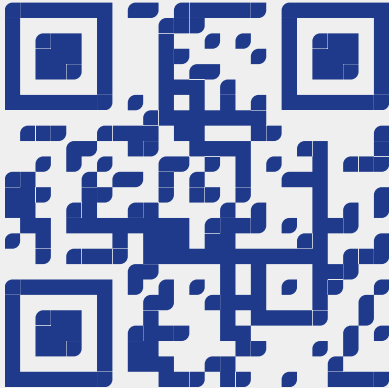
- Allow .pdf output with the same formatting as Word
- Enable login, saved reports, and version history
- Visual variable placement for non-technical users
- Let users adjust color, layout, labels before generation
- Shared library of reusable Word templates

reportGen Demonstration



Q & A

Feedback Form



THANKS FOR YOUR LISTENING!

